# FitBuddy AI Fitness Plan Generator

## Complete Project Report

FastAPI Gemini SQLite SQLAlchemy Jinja2

| Field | Details |
|---|---|
| **Project** | FitBuddy AI Fitness Plan Generator |
| **Category** | AI Assisted Web Application |
| **Submission** | Complete Source Code and Phase Documentation |
| **Prepared** | October 2026 |

Academic project documentation

# Abstract

FitBuddy is an AI-assisted fitness planning web application that generates a personalized seven-day workout schedule from a user's age, weight, goal, and preferred intensity. The application also provides a concise nutrition or recovery tip, accepts user feedback to revise the latest plan, stores original and revised plans in SQLite, and exposes an admin view plus JSON API endpoints.

The solution uses FastAPI, Pydantic, Jinja2, SQLAlchemy, SQLite, and the Google Gen AI Python SDK. A deterministic offline fallback ensures that required project flows remain demonstrable without a live key or during a temporary API failure. The verified build includes nine passing automated tests with 91 percent application-code coverage, phase-wise documents, screenshots, an editable report, a PDF, and a five-minute demonstration script.

# Contents

1 Introduction
2 Objectives and Scope
3 Requirements Analysis
4 System Design
5 Database and API Design
6 Development and Implementation
7 User Interface and Working Output
8 Testing and Results
9 Security Safety and Limitations
10 Installation and Operation
11 Demonstration and Submission
12 Conclusion
References
Appendices

# 1 Introduction

Many beginners understand a fitness goal but struggle to convert it into a balanced weekly routine. Generic plans may not reflect age, weight, preferred intensity, recovery needs, or later feedback. FitBuddy provides a single interface that creates, saves, and revises a structured week.

The project demonstrates an end-to-end AI web application rather than a standalone model call. Validated data moves through a FastAPI route, an AI service, and a database service before a Jinja2 page displays the result. The same core services also support JSON endpoints.

## 1 1 Problem Statement

Users need a clear, accessible way to obtain a seven-day fitness schedule and refine it without manually rewriting the entire plan. Students also need a project that reliably demonstrates AI integration, persistence, testing, documentation, and user experience.

## 1 2 Proposed Solution

- Collect the minimum profile data required for a useful plan.
- Generate Day 1 through Day 7 with warm-up, main work, cooldown, and recovery.
- Create a goal-related nutrition and hydration tip.
- Use feedback to produce and store a revised version.
- Display users and plan status in a coach or evaluator view.
- Remain demonstrable through a transparent offline fallback.

# 2 Objectives and Scope

## 2 1 Objectives

- Build a complete FastAPI application with a responsive web interface.
- Use Gemini through the current Google Gen AI SDK with a configurable model.
- Persist users, original plans, nutrition tips, feedback, and revised plans.
- Expose both HTML workflows and validated JSON endpoints.
- Provide automated evidence for required functionality.
- Package source code, eight phases, report, screenshots, and demo guidance.

## 2 2 Scope

| Included | Excluded |
|---|---|
| Plan generation and feedback revision | Medical diagnosis or treatment |
| Five supported fitness goals | Wearable device integration |
| Three intensity levels | Payments or subscriptions |
| SQLite storage and admin page | Public user authentication |
| Gemini and offline fallback modes | Cloud production deployment |
| Tests and formal documentation | Prescription nutrition plans |

# 3 Requirements Analysis

Requirements were derived from the supplied project description and demonstration instructions. Each major functional requirement maps to an implemented route or test.

| ID | Functional requirement | Acceptance condition |
|---|---|---|
| FR 01 | Collect user profile and preference | Six required inputs are validated |
| FR 02 | Generate weekly plan | Output contains Day 1 through Day 7 |
| FR 03 | Generate nutrition tip | Tip is shown on result page |
| FR 04 | Persist user and original plan | SQLite records are created |
| FR 05 | Revise from feedback | Updated plan is stored and displayed |
| FR 06 | Display admin view | User and latest status are listed |
| FR 07 | Provide API endpoints | JSON creation and feedback succeed |
| FR 08 | Support demo mode | Full flow works without an API key |

## 3 1 Nonfunctional Requirements

| Area | Design response |
|---|---|
| Usability | Responsive form, clear labels, readable status and errors |
| Security | Environment-only key, ignored secret files, validated input |
| Reliability | Fallback mode and controlled error responses |
| Maintainability | Separated configuration, routes, AI, services, and database |
| Testability | Deterministic output and isolated test database |
| Safety | General guidance and explicit pain/health warning |

# 4 System Design

FitBuddy follows a layered design. The browser interacts with HTML routes, Pydantic validates incoming data, the AI service generates content, service functions manage persistence, and SQLAlchemy maps data to SQLite.

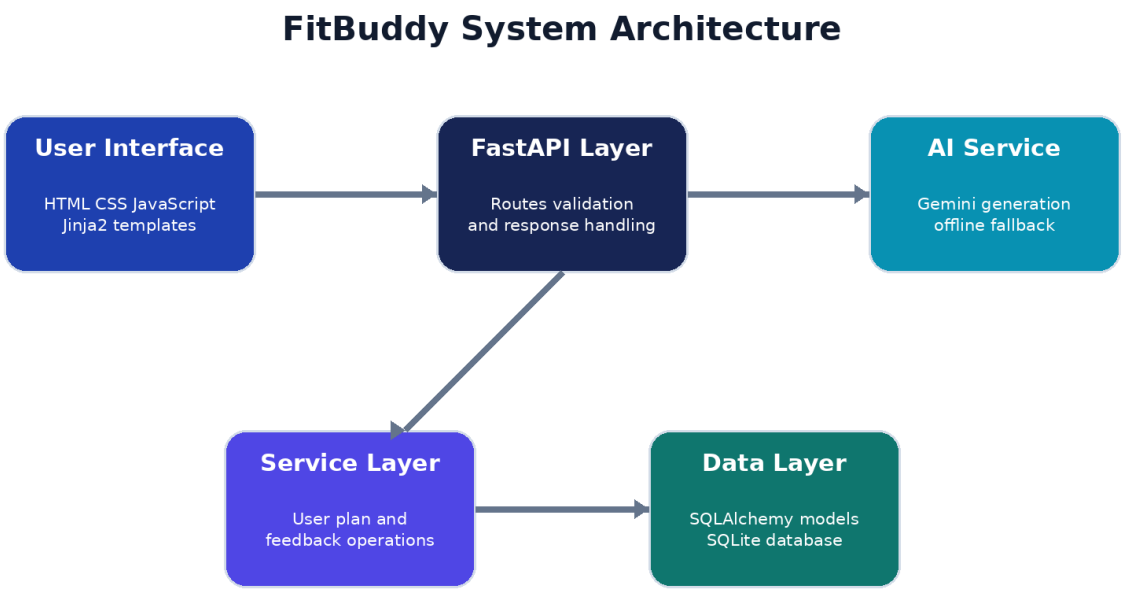



*Figure 1 FitBuddy system architecture*

## 4 1 Component Design

| Module | Responsibility |
|---|---|
| config.py | Environment variables and AI mode |
| database.py | Engine sessions entities and table creation |
| schemas.py | Validated workout feedback and response models |
| ai_service.py | Gemini prompts and deterministic fallback |
| services.py | User plan feedback and list operations |
| routes.py | HTML pages and JSON endpoints |
| main.py | FastAPI application startup and static mounting |

# 5 Database and API Design

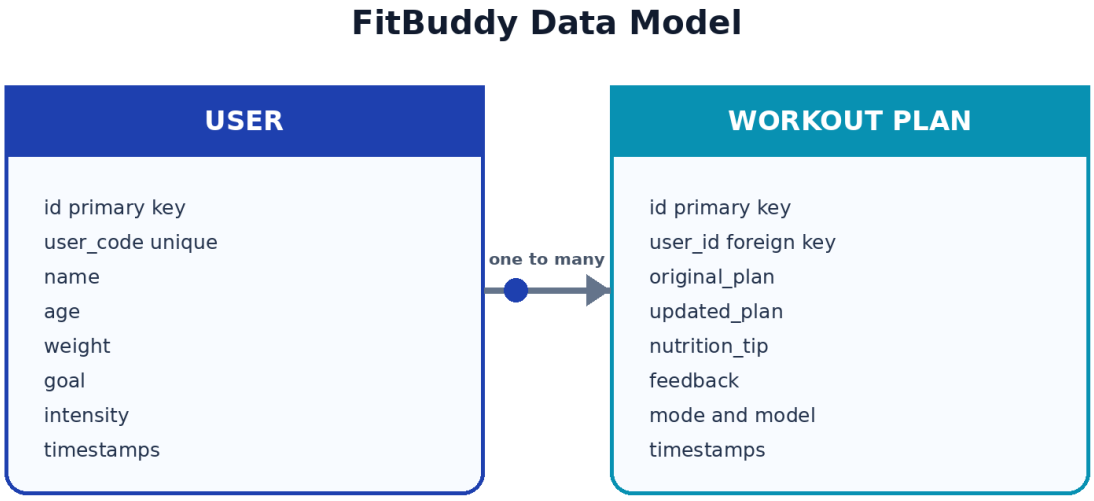



*Figure 2 FitBuddy data model*

## 5 1 Main Routes

| Method | Path | Purpose |
|---|---|---|
| GET | / | Display plan form |
| POST | /generate-workout | Generate save and display plan |
| POST | /submit-feedback | Revise latest plan |
| GET | /view-all-users | Display coach dashboard |
| GET | /api/health | Return application and AI status |
| POST | /api/workouts | Create plan as JSON |
| POST | /api/feedback | Revise plan as JSON |
| GET | /docs | Open interactive API documentation |

## 5 2 Input Validation

| Field | Rule |
|---|---|
| Name | 2 to 80 characters |
| User ID | 2 to 40 characters |
| Age | 13 to 90 |

| Field | Rule |
|---|---|
| Weight | 25 to 300 kilograms |
| Goal | One of five supported values |
| Intensity | Low medium or high |
| Feedback | 5 to 500 characters |

# 6 Development and Implementation

The project uses the current `google-genai` package. The model is read from `GEMINI_MODEL`, so the application can be updated without source changes. The default supplied configuration uses `gemini-3.8-flash`.

## 6 1 Application Startup

```
app = FastAPI(title=settings.app_name, version=settings.app_version, lifespan=lifespan)
app.mount('/static', StaticFiles(directory=STATIC_DIR), name='static')
app.include_router(router)
```

## 6 2 Gemini Request

```
with genai.Client(api_key=settings.gemini_api_key) as client:
    response = client.models.generate_content(
        model=settings.gemini_model,
        contents=prompt,
    )
```

## 6 3 Resilient Generation

If no key is configured, demo mode is enabled, or Gemini fails while fallback is allowed, the AI service returns a deterministic seven-day plan. Every stored plan records the model and generation mode, preventing the demonstration from presenting fallback output as a live Gemini response.

## 6 4 Security Controls

- `.env` is excluded from Git while `.env.example` shows safe configuration names.
- All form and API input passes through Pydantic.
- SQLAlchemy parameterizes database work.
- Jinja2 escapes inserted text by default.
- API keys never enter templates, logs, tests, screenshots, or saved records.

# 7 User Interface and Working Output

The interface uses a responsive navy, blue, cyan, and violet design system. The same application responses used during verification produced the screenshots below.

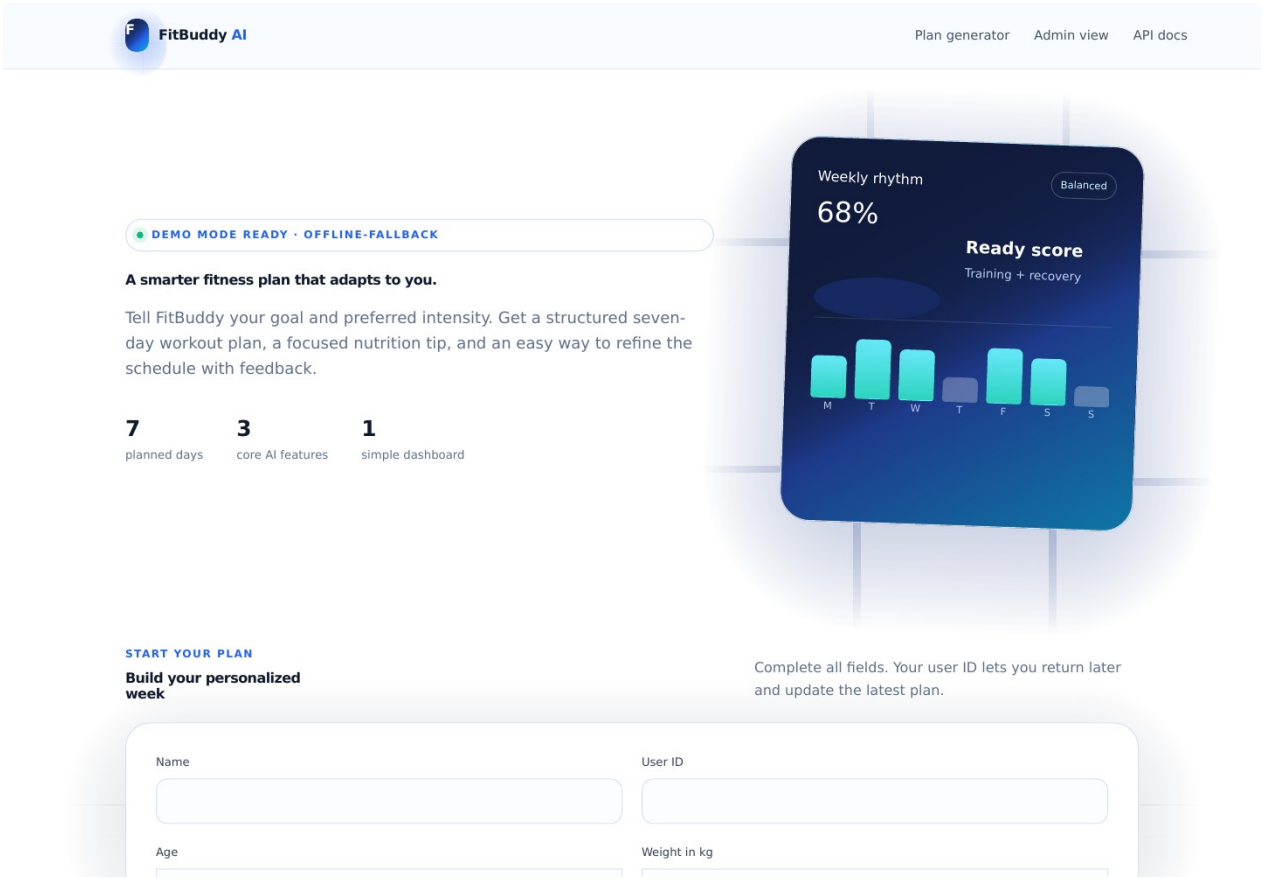



*Figure 3 Plan generator home page*

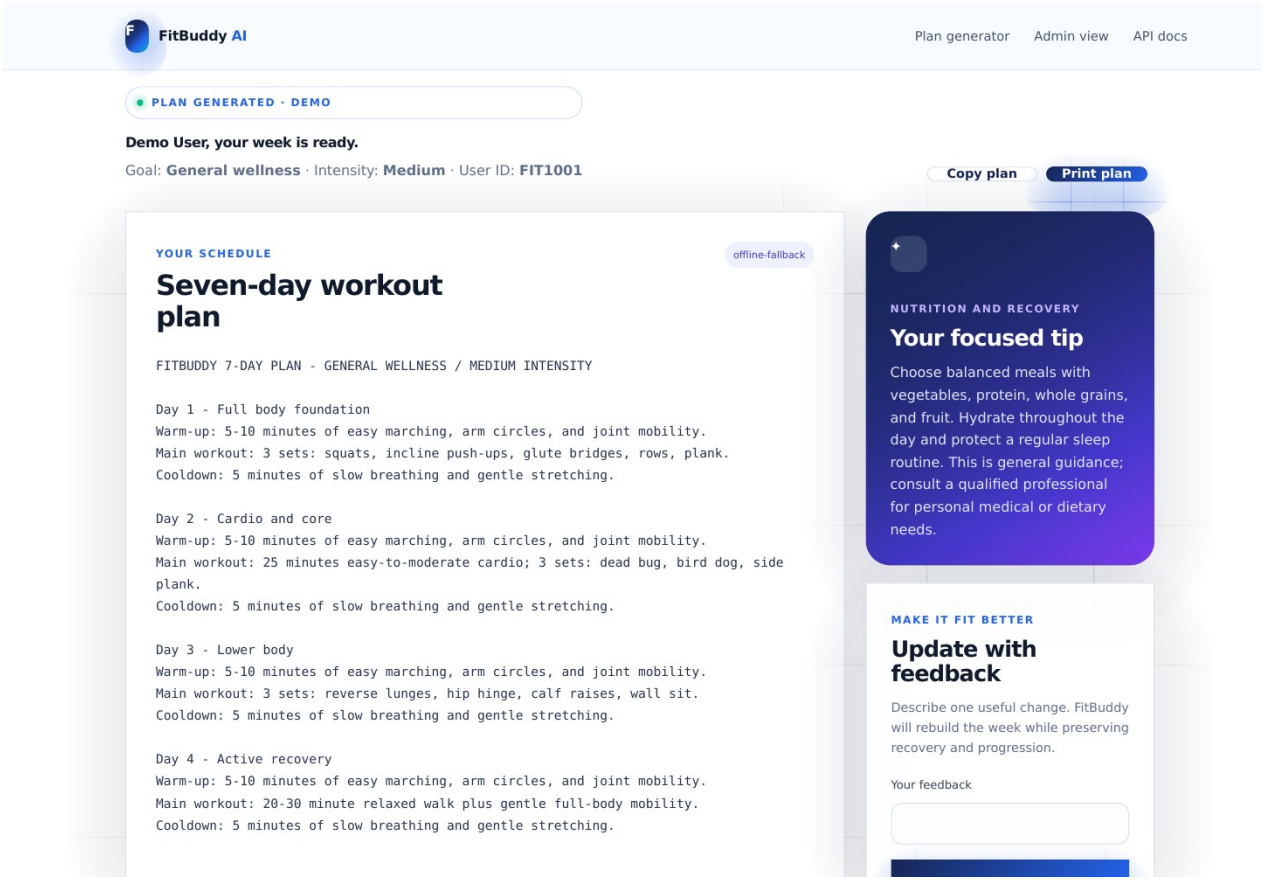



*Figure 4 Generated plan and nutrition tip*

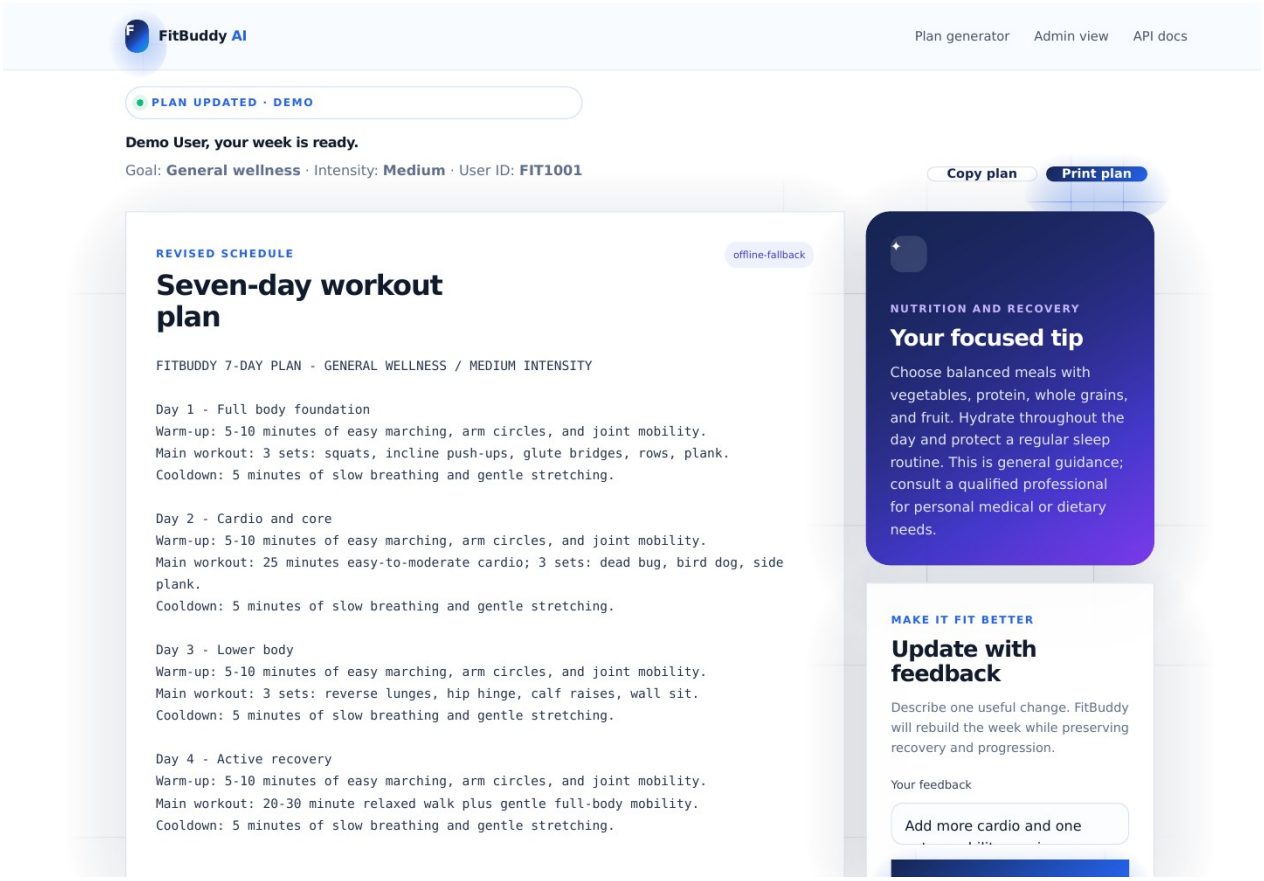



*Figure 5 Feedback based plan update*

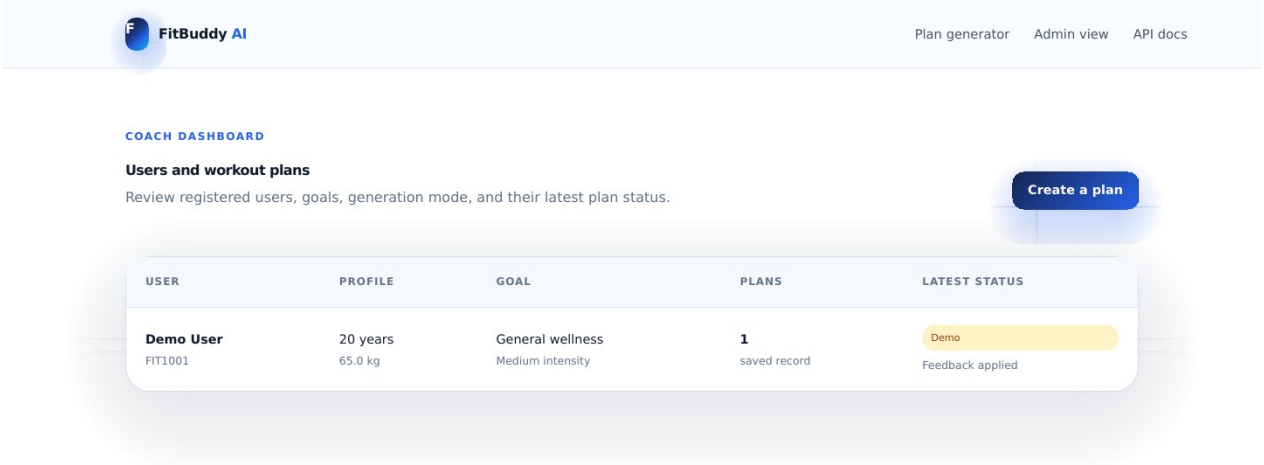



*Figure 6 Admin dashboard*

# 8 Testing and Results

Automated testing ran in demo mode with a separate SQLite database. This proves the required web, database, validation, feedback, admin, and API flows without consuming a live key.

| Metric | Verified result |
|---|---|
| Tests | 9 passed |
| Failures | 0 |
| Application code coverage | 91 percent |
| Database | Isolated SQLite test database |
| External AI calls | None during tests |

## 8 1 Test Cases

| ID | Scenario | Result |
|---|---|---|
| TC 01 | Home page and demo status | Pass |
| TC 02 | Generate and store a seven-day plan | Pass |
| TC 03 | List user on admin page | Pass |
| TC 04 | Update latest plan from feedback | Pass |
| TC 05 | Reject unknown feedback user | Pass |
| TC 06 | Reject invalid age | Pass |
| TC 07 | JSON create and update flow | Pass |
| TC 08 | Health endpoint | Pass |
| TC 09 | Fallback seven-day content and safety | Pass |

```
python -m pytest --cov=app --cov-report=term-missing
.........  [100%]
9 passed
TOTAL application coverage 91%
```

# 9 Security Safety and Limitations

## 9 1 Security

- Secret values stay in environment variables.
- Input ranges and lengths reduce malformed requests.
- Database operations use an ORM.
- The repository ignores secrets, local environments, caches, and databases.

## 9 2 Safety

FitBuddy provides general wellness information, not medical diagnosis or treatment. Prompts and fallback plans include a safety note. Users should stop if they feel sharp pain and seek qualified guidance for injuries or health concerns.

## 9 3 Limitations

- AI output can vary and may need professional review.
- The academic admin page does not include authentication.
- SQLite is suitable for local demonstration but not every large public deployment.
- The application does not integrate wearables or clinical data.
- A public production service would require formal privacy controls and rate limits.

# 10 Installation and Operation

1. Install Python 3.11 or newer.
2. Open the FitBuddy AI project folder in VS Code.
3. Create and activate a virtual environment.
4. Install requirements.
5. Copy `.env.example` to `.env` and add a private key if live Gemini is required.
6. Start Uvicorn and open the local URL.

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

## 10 1 Main URLs

| URL | Use |
| --- | --- |
| http://127.0.0.1:8000 | Plan generator |
| http://127.0.0.1:8000/view-all-users | Admin dashboard |
| http://127.0.0.1:8000/docs | Interactive API documentation |
| http://127.0.0.1:8000/api/health | Status check |

# 11 Demonstration and Submission

The recommended demonstration lasts approximately five minutes and includes the code structure, local startup, plan generation, nutrition tip, feedback update, admin view, and passing tests.

| Time | Demonstration activity |
|---|---|
| 0 00 to 0 30 | Name purpose and technology stack |
| 0 30 to 1 45 | Project tree core code and secure configuration |
| 1 45 to 2 15 | Start Uvicorn |
| 2 15 to 3 05 | Generate a seven-day plan |
| 3 05 to 3 40 | Show plan and nutrition tip |
| 3 40 to 4 15 | Apply feedback |
| 4 15 to 4 35 | Show admin page |
| 4 35 to 5 00 | Show tests and conclude |

## 11 1 Submission Items

- Public GitHub repository containing source, tests, docs, screenshots, and reports.
- Google Drive demo video link set to Anyone with the link Viewer.
- Phase-wise documentation and final report.
- SkillWallet tasks moved to To Be Reviewed after links are submitted.

# 12 Conclusion

FitBuddy meets the required project goal by delivering a complete AI-assisted fitness planning application rather than an isolated prototype. It generates and stores a seven-day plan, provides a supporting tip, revises the plan from feedback, displays user status, and exposes tested API endpoints.

The modular code, configurable model, secret-safe setup, deterministic fallback, passing tests, screenshots, and phase-wise documentation make the project ready for local demonstration and GitHub submission. Future improvements can add authentication, production deployment, professional content review, progress tracking, and wearable integration.

# References


1. Google AI for Developers. Gemini API documentation. https://ai.google.dev/gemini-api/docs
2. Google. Google Gen AI Python SDK documentation. https://googleapis.github.io/python-genai/
3. FastAPI. Official documentation. https://fastapi.tiangolo.com/
4. SQLAlchemy. Official documentation. https://docs.sqlalchemy.org/
5. Pydantic. Official documentation. https://docs.pydantic.dev/
6. Python Software Foundation. Python documentation. https://docs.python.org/3/

# Appendix A Project File Inventory

| Folder or file | Purpose |
| --- | --- |
| app | Application source templates and static assets |
| tests | Automated test suite |
| docs | Eight phases API guide script checklist and screenshots |
| reports | Editable and fixed-view reports |
| requirements.txt | Runtime packages |
| requirements-dev.txt | Testing packages |
| .env.example | Secret-safe configuration template |
| README.md | Main setup and submission guide |
| run.bat and run.sh | Convenient local startup |
| Dockerfile | Optional container startup |

# Appendix B Submission Checklist

- Public repository link works without sign-in.
- No `.env`, API key, virtual environment, or database is uploaded.
- All nine tests pass.
- DOCX and PDF reports are present.
- Demo video includes screen sharing and voice-over.
- Drive permission is Anyone with the link Viewer.
- GitHub documentation and video links are submitted in SkillWallet.
- Completed tasks are moved to To Be Reviewed.